

MEMORIA TÉCNICA DEL PROYECTO: SISTEMA DE GESTIÓN VETERINARIA

1. Introducción y Objetivos del Proyecto

1.1 Contexto del desarrollo

Actualmente, numerosas clínicas veterinarias continúan gestionando su flujo de trabajo mediante agendas físicas u hojas de cálculo. Este proyecto nace de la necesidad de modernizar y centralizar este ecosistema a través de una solución en la nube, permitiendo el acceso concurrente a la información, evitando la pérdida de historiales médicos y optimizando los procesos administrativos.

1.2 Objetivos Generales

Desarrollar una Aplicación Web Integral (Full-Stack) orientada a servicios que permita automatizar las tareas principales de una clínica veterinaria: gestión de altas, reserva de citas médicas, seguimiento del historial clínico de los pacientes y mantenimiento del almacén médico.

1.3 Objetivos Específicos

- Implementar un sistema de roles diferenciados para separar la lógica administrativa del portal de clientes.
- Construir una API robusta y segura que garantice la integridad y persistencia de datos.
- Diseñar una interfaz de usuario dinámica (SPA) e intuitiva que reduzca el tiempo requerido para ingresar información médica.
- Conectar las consultas o tratamientos médicos con un inventario real de productos para mantener un control de stock automático.

2. Análisis de Requisitos

2.1 Identificación de Usuarios

- **Administrador / Veterinario (User):** Personal interno de la clínica. Tiene permisos totales sobre el sistema para gestionar clientes, modificar el stock, consultar el calendario general, escribir diagnósticos y generar facturas.
- **Cliente (Owner):** Dueño de mascotas. Dispone de un área de acceso limitado para consultar su información, el estado de sus animales y facturación.

2.2 Requisitos Funcionales (RF)

- **RF01 - Gestión de Identidad:** El sistema debe permitir el inicio de sesión y autenticación segura mediante tokens de acceso.
- **RF02 - Módulo de Perfiles:** El sistema debe permitir realizar las operaciones CRUD (Crear, Leer, Actualizar y Eliminar) sobre los registros tanto de Clientes como de sus Mascotas.
- **RF03 - Agenda Médica:** El sistema debe proveer una vista de calendario interactivo para asignar y gestionar citas por paciente y fecha.

- **RF04 - Gestión de Inventario:** Debe existir un control de stock de productos y medicamentos de la clínica.
- **RF05 - Historial Clínico Integral:** El sistema debe registrar diagnósticos vinculando dichas entradas con los medicamentos del inventario para alterar automáticamente el stock correspondiente.
- **RF06 - Facturación:** El sistema debe generar registros de cobro asociados a un Cliente concreto tras la prestación del servicio médico.

2.3 Requisitos No Funcionales (RNF)

- **RNF01 - Usabilidad e Interfaz SPA:** Navegación centralizada en una única página mediante recarga dinámica, sin refrescar del navegador.
- **RNF02 - Rendimiento y Optimización:** Minimizar las consultas redundantes y saturadas a la base de datos.
- **RNF03 - Arquitectura Desacoplada:** El Frontend y Backend deben estar aislados para facilitar futuros reemplazos técnicos o un salto a entornos móviles nativos.

3. Arquitectura del Sistema y Stack Tecnológico

El proyecto se divide en dos grandes bloques comunicados bajo el protocolo HTTP enviando datos transparentes en archivo JSON.

3.1 Entorno Servidor (Backend API)

- **Framework:** Laravel (PHP). Escogido por su robustez técnica, su mapeo objeto-relacional (Eloquent ORM) y su patrón MVC.
- **Autenticación (API Security):** Laravel Sanctum, encargado de emitir y validar los tokens de acceso temporales.
- **Motor de Base de Datos:** MySQL, diseñado bajo una estructura fuertemente relacional garantizando transacciones seguras (DB::transaction).

3.2 Entorno Cliente (Frontend SPA)

- **Librería Core:** React JS, usando Vite como empaquetador para una compilación ultra rápida.
- **Estilos e Interfaz (UI):** Tailwind CSS, que provee clases base para garantizar un diseño 100% responsivo bajo métricas de *Mobile-First*.
- **Comunicación Asíncrona:** Axios, encargado de modularizar la interacción con los endpoints de la API adjuntando cabeceras de autorización y de estado de error (alertas visuales).

4. Diseño Lógico de Datos (Modelo Lógico)

La base de datos relacional aloja las siguientes entidades fundamentales y sus interrelaciones:

- **Users (Usuarios):** Almacena las credenciales de acceso de los trabajadores.
- **Owners (Dueños/Clientes):** Clientes de la clínica que abonan las facturas médicas.

- **Pets (Mascotas):** Tabla fuerte vinculada (1 a N) con Owners. Se encarga de catalogar al paciente a intervenir.
- **Appointments (Citas):** Reserva generada a raíz de un paciente agendado en un bloque de fecha.
- **Products (Inventario):** Catálogo de servicios y consumibles con cuantificador de stock numérico que pueden reducirse.
- **Medical_Records (Historial Médico):** Tabla estructural. Agrupa y vincula qué paciente fue intervenido en qué cita y qué "Inventario/Producto" consumió para su tratamiento.
- **Invoices (Facturas):** Registro económico que se enlaza al Dueño (Owner) para mantener un control de caja global.

5. Descripción de los Módulos Desarrollados del Sistema

- **Módulo de Dashboard Inicial:** Un panel asíncrono que redirige vistas u oculta permisos dependiendo del token validado por Auth Sanctum tras el logueo, permitiendo distinguir una sesión "Cliente" de una "Veterinario".
- **Módulo de Gestión de Registros:** Uso de tarjetas interactivas aisladas a través de Ventanas Modales dinámicas para agregar Dueños y Mascotas sin colisionar componentes visuales.
- **Módulo de Calendario:** Vista mensual/semanal/diaria organizada interactivamente para alta y visualización de consultas (sin sobrecarga visual de horas innecesarias).
- **Módulo Médico y de Consumibles:** Interconexión entre los apuntes clínicos del veterinario registrando un diagnóstico a la vez que automatiza una baja real de stock en la API.

6. Dificultades Técnicas Encontradas y Soluciones Aplicadas

1. **Gestión del Estado de los Componentes en React:** Durante las fases iniciales existían grandes conflictos de renderizado al desplegar información en listas masivas provocando errores visuales colaterales. *Solución implementada:* Aislamiento de variables de control (States) propulsando la información a modales de pantalla completa totalmente independientes.
2. **Cruces de Peticiones y Puertos (CORS):** El servidor de interfaz web de Vite bloqueaba transacciones de actualización en base de datos hacia el servidor backend local de Laravel por medidas de protección. *Solución implementada:* Configuración estricta de las políticas de dominios de origen (Cross-Origin) configurando reglas robustas en el Middleware de Laravel de la API.
3. **Arquitectura Relacional y Cargas Masivas:** El Frontend sufría un retardo de red excesivo al intentar cargar jerarquías profundas (Dueños, luego Mascotas, luego Citas). *Solución implementada:* Aprovechamiento absoluto del modelo ORM de Laravel despachando directivas jerárquicas en bloque en el Controlador optimizando las consultas.

7. Conclusiones

La elaboración del proyecto ha demostrado la enorme viabilidad técnica de una clínica gestionada íntegramente de manera escalable y digital. El uso de Laravel garantizó una lógica de procesos blindada e inquebrantable desde el lado servidor, mientras que incluir el ecosistema de React ha permitido una carga sin interrupciones molestas o refrescos forzados para el usuario final. La aplicación desarrollada en general simula con precisión los rigurosos estándares exigibles hoy en día en el desarrollo de software comercial e interactivo dentro del sector informático y médico-veterinario.